What is this?

This notebook is part of a collection of `pluto` notebooks on various topics discussed during the Time Domain Astrophysics course delivered by Stefano Covino at the Università dell'Insubria in Como (Italy). Please direct questions and suggestions to stefano.covino@inaf.it.

Table of Contents

TIME-DOMAIN ASTROPHYSICS

Stefano Covino
INAF / Brera Astronomical Observatory

Gaussian Processes

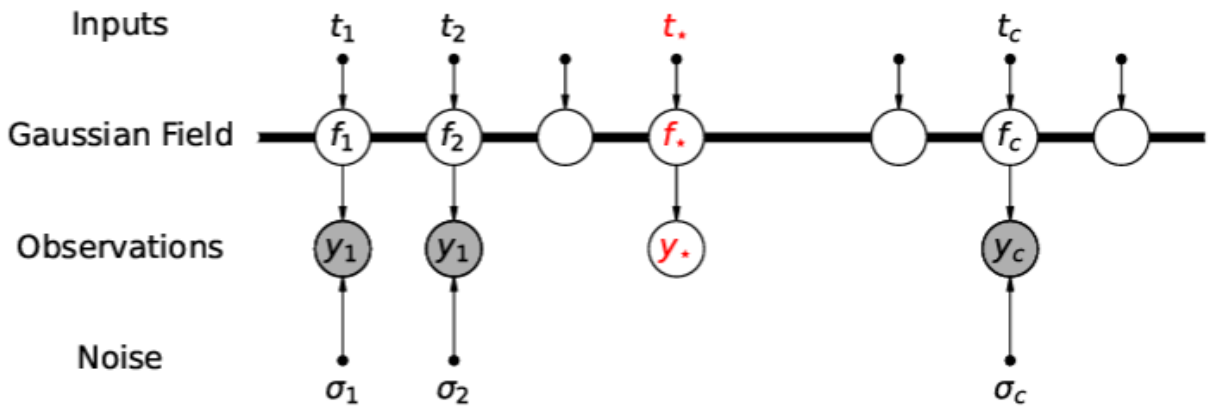
- A GP is a type of *stochastic process* based on the Gaussian probability distribution.
- The formal definition of a GP is that the joint probability distribution over any finite sample $y = \{y_i\}_{i=1,\ldots,N}$ from the GP is a multi-variate Gaussian:

$$p(y) = \mathcal{N}(m, K)$$

- where m is the mean vector and K the covariance matrix.
- The elements of the mean vector and covariance matrix are given by the mean function m and the covariance function k, respectively:

$$m_i = m(x_i, \theta), \quad K_{i,j} = k(x_i, x_j, \varphi)$$

- where x_i is the set of inputs (independent variables) corresponding to the ith sample.
- For time-series data, the inputs usually include, but aren't necessarily restricted to, the time t_i.
- The covariance function is also known as the kernel function, and it is a fundamental ingredient of a GP model.



- In the sketch above we show a probabilistic graphical model for GP regression given observation y at inputs t, and a prediction at test input t_*. Dots represent fixed variables (inputs), grey circles represent observed variables, white circls represent unknown (latent) variables. The thick horizontal bar indicates a set of fully connected nodes (meaning that every f depends on every t). On the other hand, each observed output y_i depends only on the corresponding f_i and σ_i, and is conditionally independent of the other variables. In this example, the measurement uncertainties σ and the GP hyper-parameters (not shown) are treated as known (fixed).
- The parameters θ and φ of the mean and covariance function are known as hyper-parameters of the GP.
- Strictly speaking, the parameters of the GP are the (infinitely many) unknown functions that share the specified mean vector and covariance matrix and could have given rise to the observations. However, these parameters are always marginalised over: we never explicitly deal with the individual functions.
- GPs are therefore a type of Hierarchical Bayesian Model (HBM).

- In principle, it is possible to construct and use stochastic process models based on other distributions, GPs are by far the most popular, for two main reasons:
    - The first is Central Limit theorem: it implies that the assumption of Gaussianity is often at least approximately correct.
    - The second is that Gaussian distributions obey simple mathematical identities for marginalisation and conditioning, that enable inference with GPs.

A pragmatic approach to GPs

- GP regression (GPR) can be thought of as a generalisation of least-squares regression, allowing for correlated noise (or signals) in the data.
- Conversely, least-squares regression, as traditionally presented, is a special case of GPR, where the covariance matrix is assumed to be purely diagonal, and the variances associated with each observation are known *a priori*.

Least-square Regression

- Let's consider N observations of a variable $y = \{y_i\}_{i=1,\ldots,N}$, taken at times $t = \{t_i\}$, with associated measurement uncertainties $\sigma = \{\sigma_i\}$.
- We wish to compare these to a model function $m(t, \theta)$ controlled by parameters $\theta = \{\theta_j\}_{j=1,\ldots,M}$.
- In least-squares regression, we minimize the quantity:

$$\chi^2 \equiv \sum_{i=1}^{N} (y_i - m_i)^2 / \sigma_i^2$$

- where $m_i \equiv m(t_i, \theta)$, with respect to θ.
- The χ^2 minimization come from the following considerations:
- Let us assume that the observations are given by $y_i = m(t_i, \theta) + \epsilon_i$, where ϵ_i is the measurement error, or noise, on the i^{th} observation.
- Let us also assume that ϵ_i is drawn from a Gaussian distribution with mean 0 and variance σ_i^2:

$$p(\epsilon_i) = \mathcal{N}(0, \sigma_i^2) \equiv \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{\epsilon_i^2}{2\sigma_i^2}\right)$$

- Then the likelihood for the i^{th} observation is simply:

$$\mathcal{L}_i(\theta) \equiv p(y_i|\theta) = \mathcal{N}(m_i, \sigma_i^2) = \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left[-\frac{(y_i - m_i)^2}{2\sigma_i^2}\right]$$

- We also assume that the noise is uncorrelated, or white, meaning that the ϵ_i's are drawn independently of each other from their respective distributions.
- Then, the likelihood for the whole dataset y is merely the product of the likelihoods for the individual observations:

$$\mathcal{L}(\theta) \equiv p(\mathbf{y}|\theta) = \prod_{i=1}^{N} \mathcal{L}_i = \prod_{i=1}^{N} \left\{ \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left[-\frac{(y_i - m_i)^2}{2\sigma_i^2}\right] \right\}$$

- One can readily see that $\ln \mathcal{L} = \text{constant} - 0.5\chi^2$, where the constant depends only on the σ's.
- Thus, if the σ's are known, maximizing $\mathcal{L}$ is equivalent to minimizing χ^2.
- In other words, least-squares regression yields the Maximum Likelihood Estimate (MLE) of the parameters under the assumption of white, Gaussian noise with known variance.

From least square regression to GPs

- Let us now re-write the likelihood in matrix form:

$$\mathcal{L}(\theta, \phi) = \frac{1}{\sqrt{2\,\pi\,|\mathbf{K}|}} \exp\left(-\tfrac{1}{2}(\mathbf{y}-\mathbf{m})^{\mathrm{T}}\mathbf{K}^{-1}(\mathbf{y}-\mathbf{m})\right)$$

- where, the mean vector $\mathbf{m}$ has elements $m_i = m(t_i, \theta)$ and $\mathbf{K}$ is a purely diagonal (N, N) *covariance matrix* of the model with elements $K_{ij} = \delta_{ij}\sigma_i^2$ (δ_{ij} being the discrete Kronecker delta function).
- Let us allow a more flexible covariance model:

$$K_{ij} = k(t_i, t_j, \phi) + \delta_{ij}\sigma_i^2$$

- where k is a *covariance function*, or *kernel function*, controlled by parameters ϕ.
- Depending on the choice of kernel function and parameters, the covariance matrix can now have non-zero off-diagonal elements, allowing us to explicitly model correlated noise or stochastic signals in the data.
- The kernel function k encodes our beliefs about the stochastic, or random, element of the model, in just the same way as the mean function m encodes our beliefs about the deterministic component of the model.

Common kernel functions

- The kernel function can be any positive scalar function that gives rise to a positive semi-definite covariance matrix over the input domain.
- Some popular kernel functions are listed below:

Name	Representation[a]
Constant	α^2
Squared Exponential[b]	$e^{-(\tau/\lambda)^2/2}$
Exponential[c]	$e^{-\tau/\lambda}$
Matérn-3/2	$\left(1+\sqrt{3}\tau/\lambda\right)e^{-\sqrt{3}\tau/\lambda}$
Matérn-5/2	$\left(1+\sqrt{5}\tau/\lambda+5(\tau/\lambda)^2/3\right)e^{-\sqrt{5}\tau/\lambda}$
Rational quadratic	$\left(1+\frac{\tau^2}{2\gamma\lambda^2}\right)^{-\gamma}$
Cosine	$\cos 2\pi\tau/\lambda$
Sine Squared Exponential	$\exp\left(-\Gamma\sin^2\pi\tau/\lambda\right)$
Stochastic Harmonic Oscillator[d]	$\cos\left(\sqrt{1-\beta^2}\frac{\tau}{\lambda}\right)+\frac{\beta}{\sqrt{1-\beta^2}}\sin\left(\sqrt{1-\beta^2}\frac{\tau}{\lambda}\right)$

[a] in each case, τ is defined as $\tau = |t_i - t_j|$, and Greek letters indicate hyper-parameters; [b] "radial basis function"; [c] "Ornstein–Uhlenbeck", "damped random walk" or "Matérn-1/2"; [d] Foreman-Mackey et al. (2017).

- More kernel functions are often constructed using products and sums of the standard kernels.
- Besides, addition and multiplication, other operations can be used to impose structure on the standard kernels.
    - For example, linear operations like scalar multiplication, more general affine transformations, differentiation, or integration can all be used to develop new kernel functions.
- A general rule is that the error associated to observation j is typically larger (or even much larger) than its variance due to the covariance matrix (e.g. red-noise) contribution.

Making predictions by GPs

- Given some existing observations $\mathbf{y}$, taken at times $\mathbf{t}$, we can make predictions at some new set of times $\mathbf{t}_\star$, i.e. computing $p(\mathbf{y}_\star|\mathbf{y})$, the *conditional* probability distribution for $\mathbf{y}_\star$ given $\mathbf{y}$.
- This is often called the *predictive distribution*, because it is often used to extrapolate a time-series dataset forwards.
- The predictive distribution is also Gaussian, and its mean and covariance are given by simple analytic relations:

$$\mathbf{f}_\star = \mathbf{m}_\star + \mathbf{K}_\star^\mathrm{T}\,\mathbf{K}^{-1}\,(\mathbf{y} - \mathbf{m}) \qquad \mathbf{C}_\star = \mathbf{K}_{\star\star} - \mathbf{K}_\star^\mathrm{T}\,\mathbf{K}^{-1}\,\mathbf{K}_\star$$

- where $\mathbf{m}_\star \equiv m(\mathbf{t}_\star;\theta)$, $[\mathbf{K}_\star]_{ij} = k(t_i, \mathbf{t}_{\star,j};\phi)$ and $[\mathbf{K}_{\star\star}]_{ij} = k(\mathbf{t}_{\star,i}, \mathbf{t}_{\star,j};\phi)$.

Exercise: GPs from scratch

- There are plenty of highly optimized libraries for GP inference. Yet, it is instructive to write a GP code starting from the beginning.
- A common applied statistics task involves building regression models to characterize non-linear relationships between variables. It is possible to fit such models by assuming a particular non-linear functional form, such as a sinusoidal, exponential, or polynomial function, to describe one variable's response to the variation in another. Unless this relationship is obvious from the outset, however, it involves possibly extensive model selection procedures to ensure the most appropriate model is retained. Alternatively, a non-parametric approach can be adopted by defining a set of knots across the variable space and use a spline or kernel regression to describe arbitrary non-linear relationships. However, knot layout procedures are somewhat ad hoc and can also involve variable selection. A third alternative is to adopt a Bayesian non-parametric strategy, and directly model the unknown underlying function. For this, we can employ Gaussian process models.
- Describing a Bayesian procedure as "non-parametric" is something of a misnomer. The first step in setting up a Bayesian model is specifying a full probability model for the problem at hand, assigning probability densities to each model variable. Thus, it is difficult to specify a full probability model without the use of probability functions, which are parametric. In fact, Bayesian non-parametric methods do not imply that there are no parameters, but rather that the number of parameters grows with the size of the dataset. Rather, Bayesian non-parametric models are infinitely parametric.
- Let's assume that our data can be modeled by a Gaussian distributions:

$$p(x \mid \pi, \Sigma) = (2\pi)^{-k/2}|\Sigma|^{-1/2}\exp\left\{-\frac{1}{2}(x-\mu)'\Sigma^{-1}(x-\mu)\right\}$$

- Gaussians distributons own a set of very useful mathematical properties, e.g., the marginal distribution of any subset of elements from a multivariate normal distribution is also normal:

$$p(x,y) = N\left(\begin{bmatrix}\mu_x \\ \mu_y\end{bmatrix}, \begin{bmatrix}\Sigma_x & \Sigma_{xy} \\ \Sigma_{xy}^T & \Sigma_y\end{bmatrix}\right)$$

$$p(x) = \int p(x,y)dy = N(\mu_x, \Sigma_x)$$

- A Gaussian process generalizes the multivariate normal to infinite dimension. It is defined as an infinite collection of random variables, with any marginal subset having a Gaussian distribution. Thus, the marginalization property is explicit in its definition.

    - Another way of thinking about an infinite vector is as a function. When we write a function that takes continuous values as inputs, we are essentially implying an infinite vector that only returns values (indexed by the inputs) when the function is called upon to do so. By the same token, this notion of an infinite-dimensional Gaussian represented as a function allows us to work with them computationally: we are never required to store all the elements of the Gaussian process, only to calculate them on demand.
- So, we can describe a Gaussian process as a *distribution over functions*. Just as a multivariate normal distribution is completely specified by a mean vector and covariance matrix, a GP is fully specified by a mean function and a covariance function:

$$p(x) \sim GP(m(x), k(x, x'))$$

- It is the marginalization property that makes working with a Gaussian process feasible: we can marginalize over the infinitely-many variables that we are not interested in, or have not observed.
- For example, one specification of a GP might be:

$$m(x) = 0$$

$$k(x, x') = \theta_1 exp(-\frac{\theta_2}{2}(x - x')^2)$$

- Here, the covariance function is a *squared exponential*, for which values of x and x' that are close together result in values of k closer to one, while those that are far apart return values closer to zero.
- For a finite number of points, the GP becomes a multivariate normal, with the mean and covariance as the mean function and covariance function, respectively, evaluated at those points.

Sampling from a Gaussian Process

- To make this notion of a "distribution over functions" more concrete, let's quickly demonstrate how we obtain realizations from a Gaussian process, which results in an evaluation of a function over a set of points.
- All we will do here is a sample from the prior Gaussian process, so before any data have been introduced. What we need first is our covariance function, which will be the squared exponential, and a function to evaluate the covariance at given points (resulting in a covariance matrix).

```
exponential_cov (generic function with 1 method)
```

```
function exponential_cov(x, y, prms)
    return prms[1] .* exp.( -0.5 .* prms[2] .* (x .- y').^2)
end
```

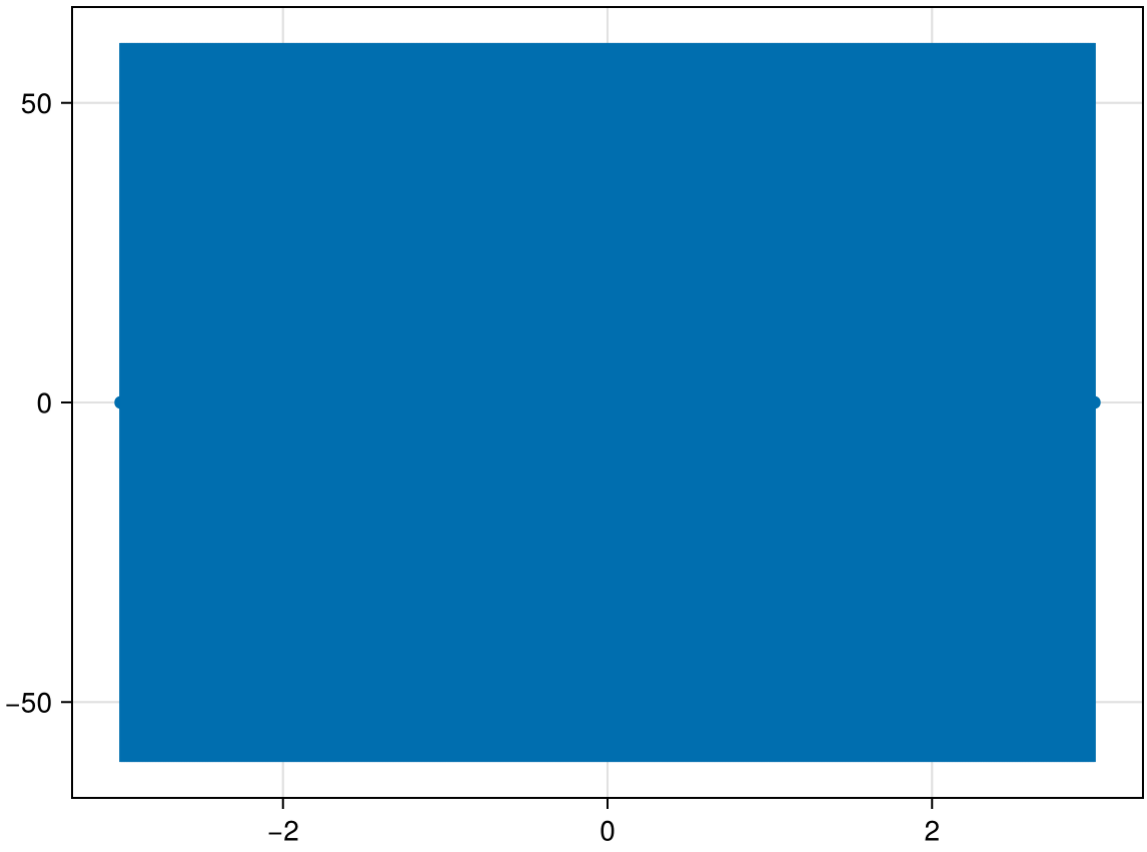
- This is instead the function implementing the conditional distribution:

```
conditional (generic function with 1 method)
```

```
function conditional(x_new, x, y, prms)
    B = exponential_cov(x_new, x, prms)
    C = exponential_cov(x, x, prms)
    A = exponential_cov(x_new, x_new, prms)
    #mu = transpose(B) * inv(C) * y
    #sigma = A - transpose(B) * inv(C) * B
    mu = transpose(inv(C) * transpose(B)) * y
    sigma = A - B * inv(C) * transpose(B)
    return(mu,sigma)
end
```

- Now, let's start with a Gaussian process prior with hyperparameters $\sigma_0 = 1, \sigma_1 = 10$. We will also assume a zero function as the mean, so we can plot a band that represents one standard deviation from the mean.

```
begin
    θ = [1, 10]
    σ0 = exponential_cov(0, 0, θ)
    xpts = range(start=-3, stop=3, step=0.01)
end;
```



- Not surprisingly, the model is just a zero-mean band with the given uncertainty.
- Things change if we begin to select an arbitrary starting point to sample, say $x = 1$.
    - Since there are no previous points, we can sample from an unconditional Gaussian:

```
begin
    d0 = Normal(0, σ0)
    x = [1.,]
    y = rand(d0,1)
    println(y)
end
```

```
[-1.5395976598223056]
```

- We can now update our confidence band, given the point that we just sampled, using the covariance function to generate new point-wise intervals, conditional on the value $[x_0, y_0]$:

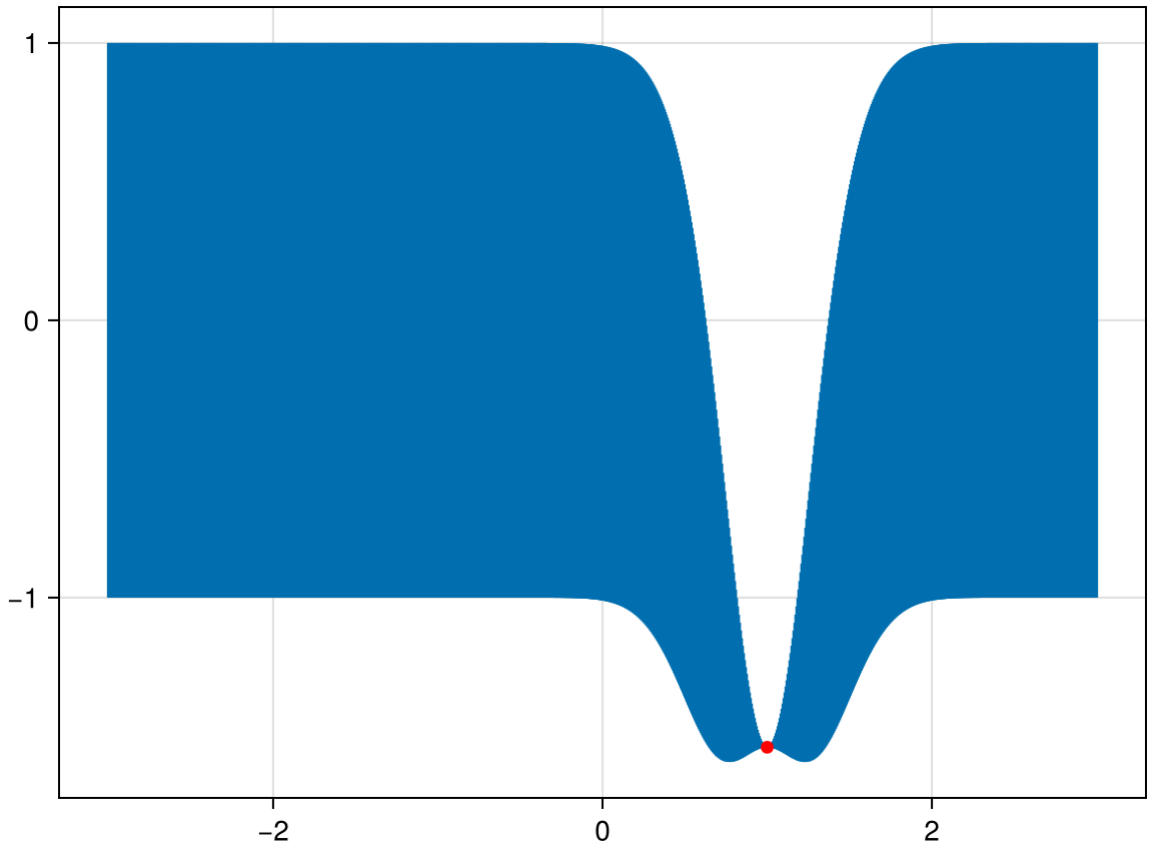
```
σ1 = exponential_cov(x, x, θ);
```

- And now let's write a function to compute the predictions of our GP:

```
predict (generic function with 1 method)
```

```
function predict(x, data, kernel, prms, sigma, t)
    k = [kernel(x, y, prms) for y in data]
    Sinv = inv(sigma)
    y_pred = transpose(k) * Sinv * t
    sigma_new = kernel(x, x, prms) - transpose(k) * Sinv * k
    return y_pred, sigma_new
end
```

```
begin
    x_pred = range(start=-3., stop=3., length=1000)
    predictions = [predict(i, x, exponential_cov, θ, σ1, y) for i in x_pred]
end;
```

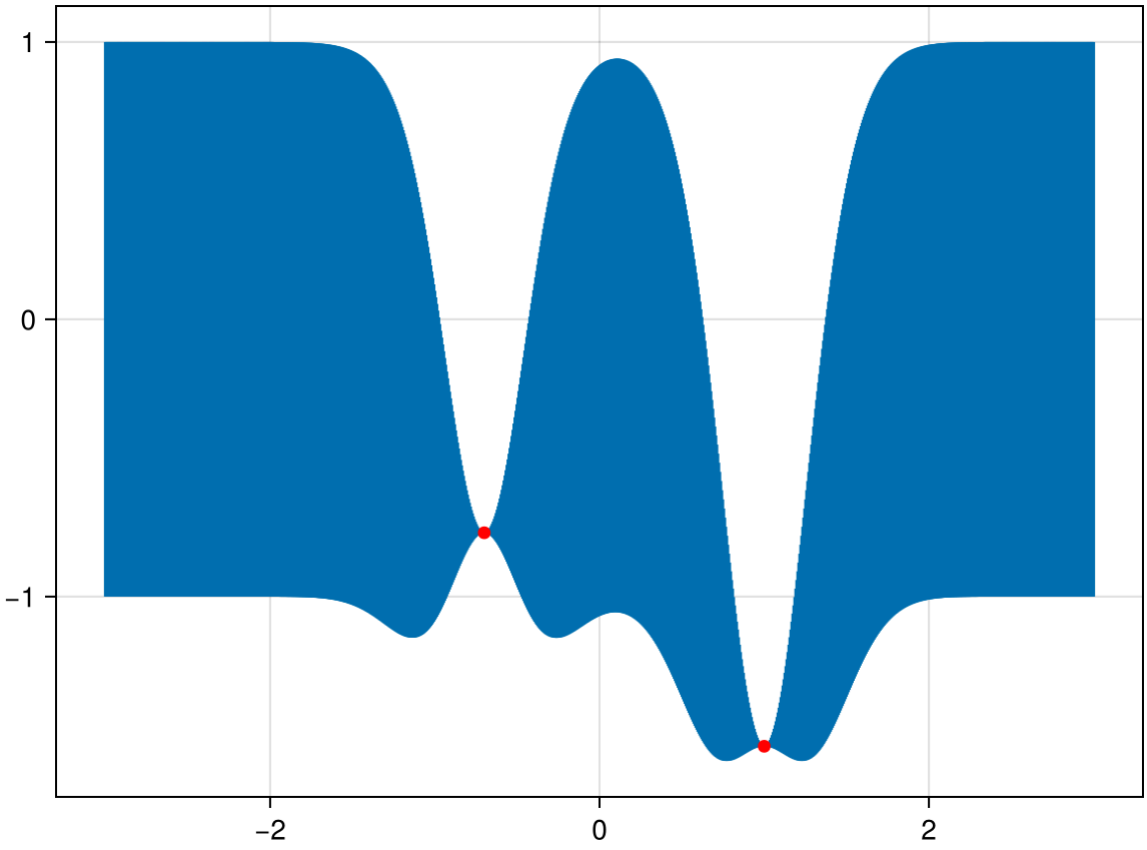


- So conditional on this point, and the covariance structure we have specified, we have essentially constrained the probable location of additional points. Let's now sample another:

```
begin
    m, s = conditional([-0.7], x, y, θ)
    d1 = Normal(m[1], s[1])
    yn = rand(d1,1)
    println(yn)
end
```

```
[-0.7697550060930933]
```

```
begin
    push!(x,-0.7)
    push!(y,yn[1])
    σ2 = exponential_cov(x, x, θ)
    predictions2 = [predict(i, x, exponential_cov, θ, σ2, y) for i in x_pred]
end;
```

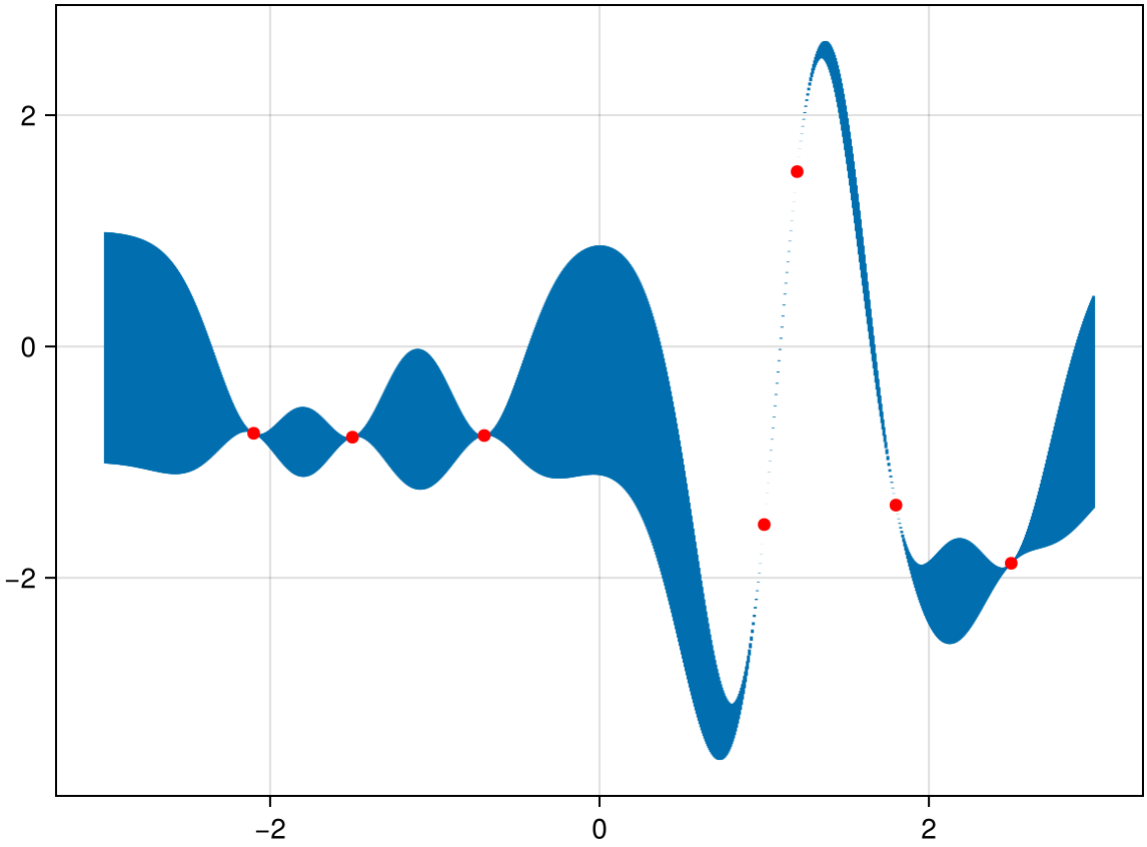


- Of course, sampling sequentially is just a heuristic to demonstrate how the covariance structure works. We can just as easily sample several points at once:

```
begin
    x_more = [-2.1, -1.5, 1.2, 1.8, 2.5]
    m_more, s_more = conditional(x_more, x, y, θ)
    dm = Normal.(m_more,s_more)
    y_more = rand(d0, length(m_more))
    println(y_more)
end;
```

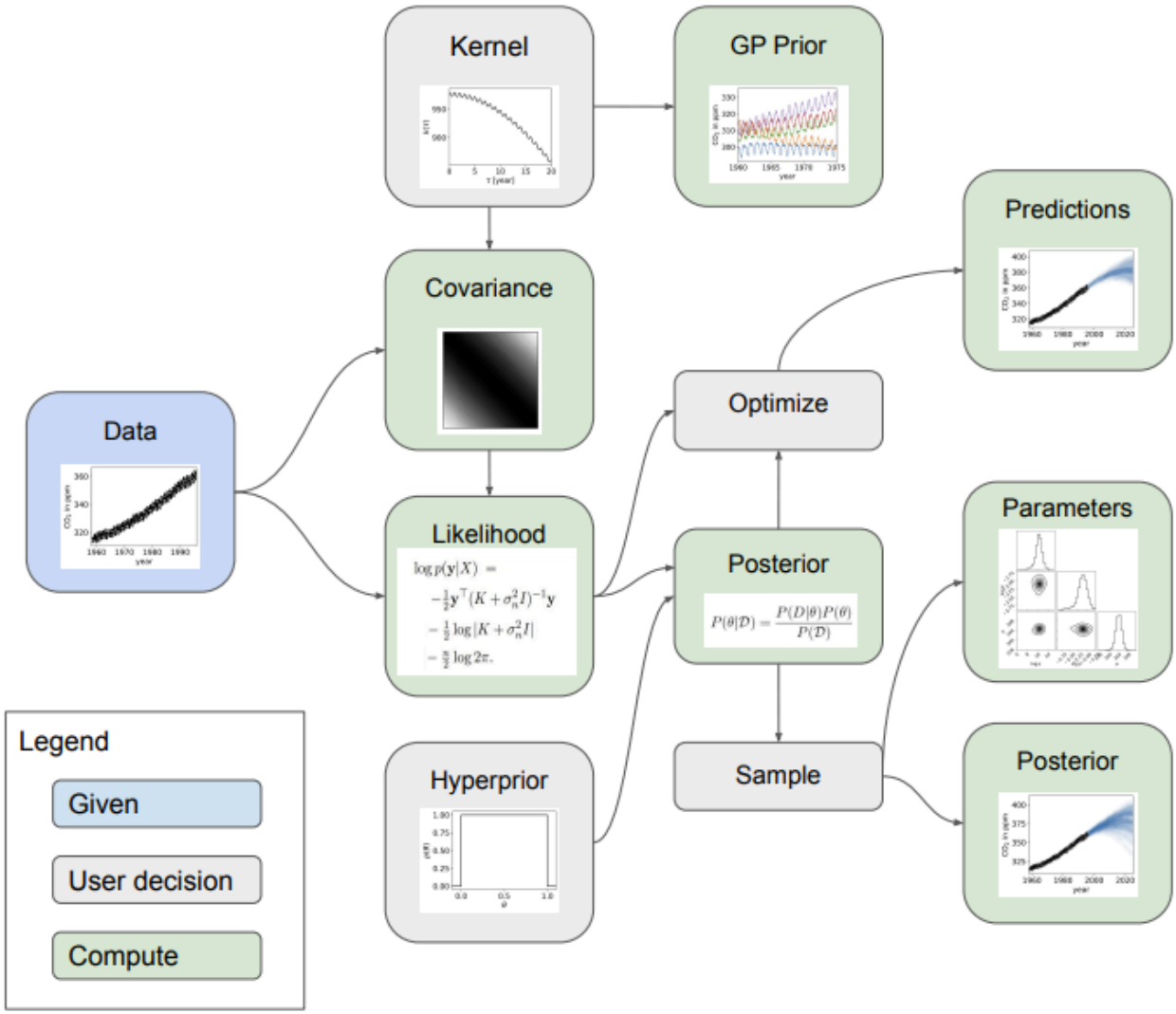
```
[-0.7503430078442939, -0.7839560338004095, 1.5120172397547167, -1.3714472076652
51, -1.8742330768245719]
```

```
begin
    xfin = vcat(x,x_more)
    yfin = vcat(y,y_more)
    σ_new = exponential_cov(xfin, xfin, θ)
    predictions_more = [predict(i, xfin, exponential_cov, θ, σ_new, yfin) for i in
    x_pred]
end;
```



- Essentially, as the density of points becomes high, it results in a realization (sample function) from the prior GP.

GPR Inference workflow



Exercize: model fitting with correlated noise

- In this example, we simulate a common data analysis situation where our dataset exhibits unknown correlations in the noise.
  - When taking data, it is often possible to estimate the independent measurement uncertainty on a single point (due to, for example, Poisson counting statistics) but there are often residual systematics that correlate data points.
  - The effect of this correlated noise can often be hard to estimate but ignoring it can introduce substantial biases into our inferences.

- The model that we analyse in this exercize is a single Gaussian feature with three parameters: amplitude α, location l, and width σ^2. This is qualitatively similar to a few problems in astronomy and physical sciences in general, e.g., fitting spectral features, measuring transit times, etc.
- Let's start by simulating a dataset of 50 points with known correlated noise using parameters: α=−1,l=0.1, and σ^2=0.4.

```
# The "feature" in our data set
#
function fundmod(x, α, l, σ2)
    return α * exp.(-(x .- l).^2 ./ (2*σ2))
end;
```

```
begin
    # A GP with "squared exponential" kernel and "fundmod" as mean function.
    # We use the Stheno package
    #
    Amp = 0.1
    Lung = 3.3
    k = Amp * with_lengthscale(SqExponentialKernel(), Lung)
    f = atomic(GP(x -> fundmod(x,-1.,0.1,0.4),k), GPC())
end;
```

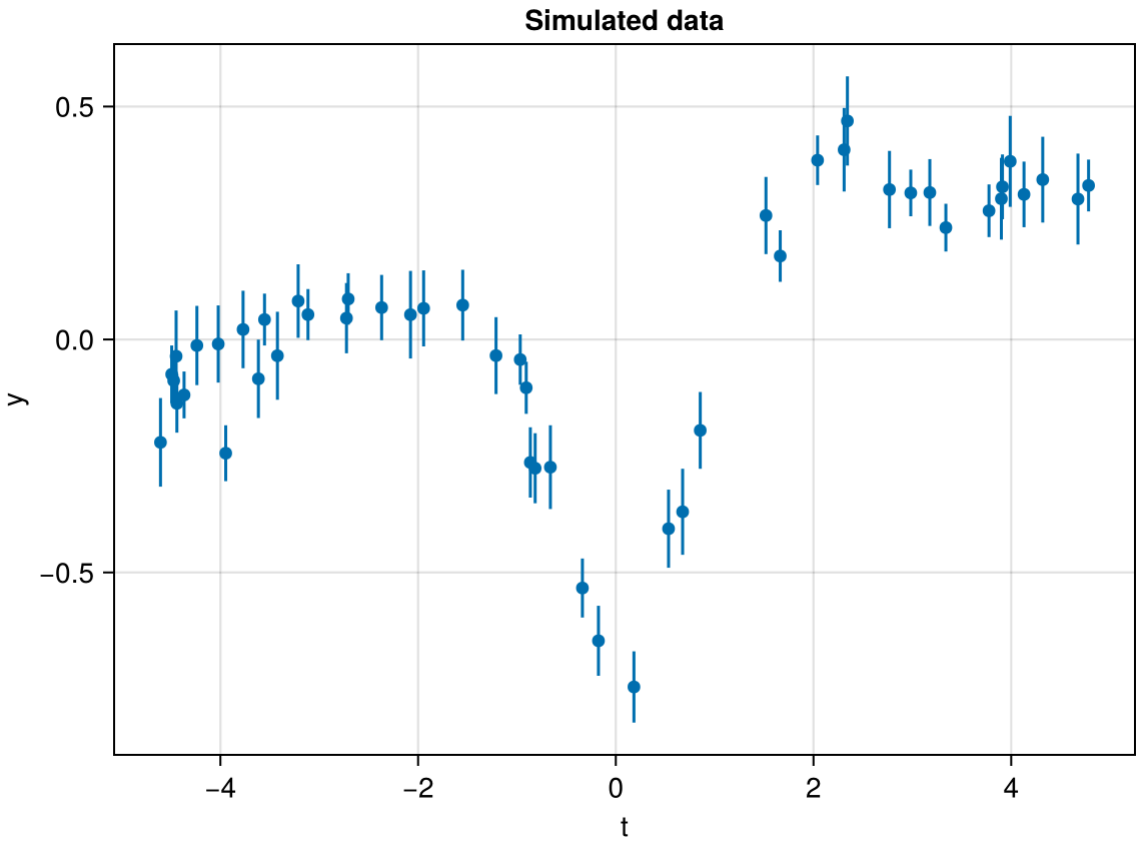
- Let's also define a seed for the random number generator for reproducibility

```
Random.seed!(6);
```

```
# The function to generate data
#
function generate_data(params, N, rng=[-5, 5])
    t = rng[1] .+ diff(rng) .* sort(rand(N))
    y = rand(f(t,1e-6))
    yerr = 0.05 .+ 0.05 .* rand(N)
    y += yerr .* randn(N)
    return t, y, yerr
end;
```

```
begin
    truth = [-1.,0.1,0.4]
    tdata, ydata, ydataerr = generate_data(truth, 50)
end;
```

- And let's see what we got:



Assuming White Noise

- Let's start by doing the standard thing and assuming that the noise is uncorrelated. In this case, the ln-likelihood function of the data given the parameters θ is:

$$\ln p(y_n|t_n,\sigma_n^2,\theta) = -\frac{1}{2}\sum_{n=1}^{N}\frac{[y_n - f_\theta(t_n)]^2}{\sigma_n^2} + A$$

- where A doesn't depend on θ so it is irrelevant for our purposes and $f_\theta(t)$ is our model function.
- It is clear that there is some sort of systematic trend in the data and we don't want to ignore that so we'll simultaneously model a linear trend and the Gaussian feature described in the previous section. Therefore, our model is:

$$f_\theta(t) = mt + b + \alpha\exp(-\frac{[t-\ell]^2}{2\sigma^2})$$

- where θ is the 5-dimensional parameter vector:

$$\theta = \{m, b, \alpha, \ell, \sigma^2\}$$

```
function fitmodel(x,θ)
    m,b,α,l,σ2 = θ
    return b .+ m .* x .+ fundmod(x, α, l, σ2)
end;
```

- And, as usual, let's define a probabilistic model for our problem. We adopt uniform priors for simplicity. In a real problem choosing adequate priors would be a fundamental step for a properly carried out analysis.

```
begin
    @model function NoKernelModel(t,y,ey)
        m ~ Uniform(-1,1)
        b ~ Uniform(-1,1)
        α ~ Uniform(-5,0)
        l ~ Uniform(-1,1)
        lσ2 ~ Uniform(-10,0)
        #
        fmod = fitmodel(t,[m,b,α,l,exp.(lσ2)])
        #
        return y ~ MvNormal(fmod,ey)
    end

    tmod = NoKernelModel(tdata,ydata,ydataerr);
end;
```

- Now, we run a MCMC chain:

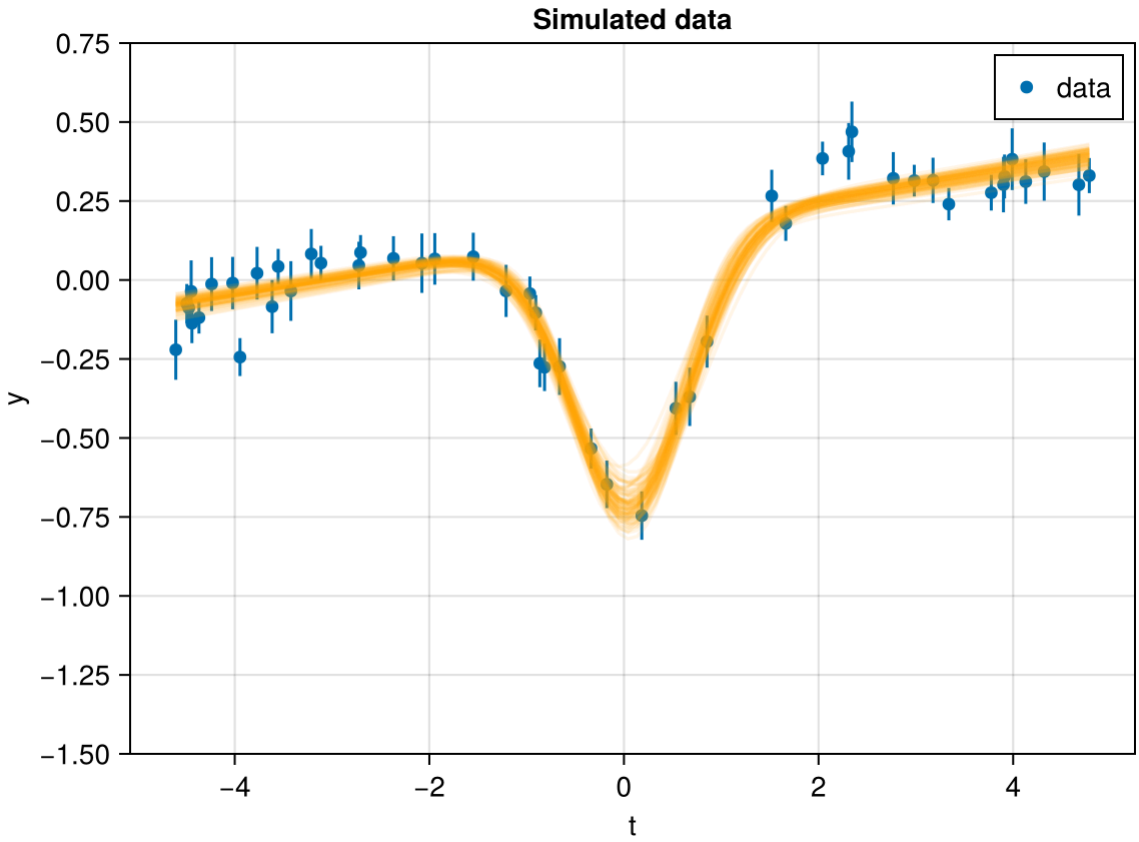
	iteration	chain	m	b	α	l	lσ2	n_steps	altro
1	1001	1	0.045573	0.162369	-0.860805	0.0591559	-0.980316	1.0	
2	1002	1	0.0531552	0.142149	-0.821293	0.0799074	-1.01276	7.0	
3	1003	1	0.046376	0.161894	-0.908529	0.0570156	-1.06147	7.0	
4	1004	1	0.0530985	0.140822	-0.862388	0.0589127	-1.13404	7.0	
5	1005	1	0.0502957	0.141061	-0.875724	0.0878591	-1.01597	7.0	
6	1006	1	0.0485508	0.157051	-0.842942	0.0671678	-1.07774	7.0	
7	1007	1	0.0426374	0.140899	-0.913199	0.0286233	-1.0302	3.0	
8	1008	1	0.0520082	0.158295	-0.922295	0.0815558	-1.20687	7.0	
9	1009	1	0.0538214	0.159791	-0.96417	0.0938305	-1.15466	3.0	
10	1010	1	0.0449098	0.120235	-0.825634	0.00683475	-1.25605	7.0	

altro

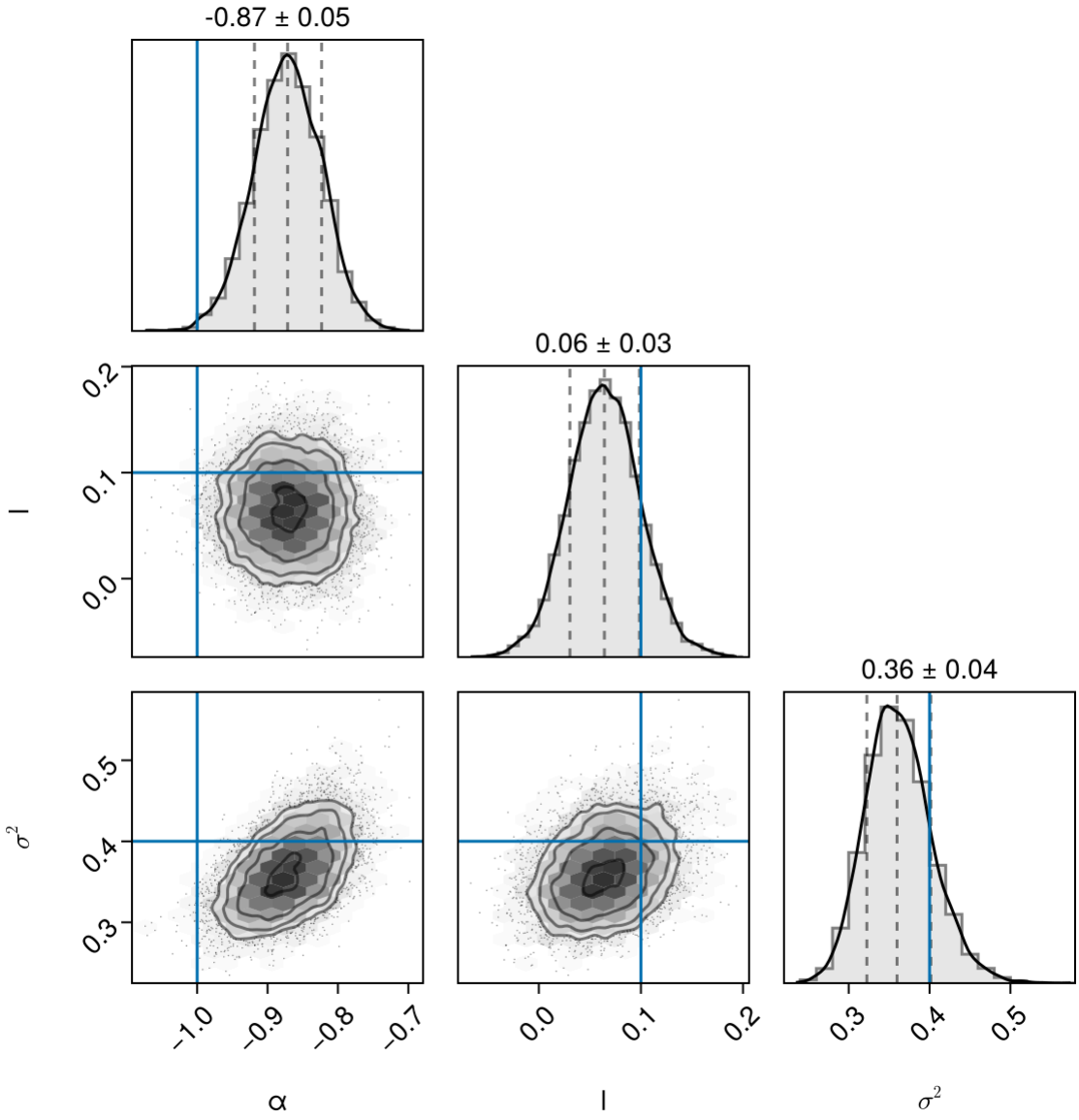
```
begin
    iterations = 2000
    burnins = 1000
    chains = 4

    chain = mapreduce(c -> sample(tmod, NUTS(burnins,0.6), iterations), chainscat,
    1:chains)
end
```

```
Sampling 100%

Found initial step size
ϵ: 0.0125

Sampling 100%

Found initial step size
ϵ: 0.0125

Sampling 100%

Found initial step size
ϵ: 0.0125

Sampling 100%

Found initial step size
ϵ: 0.0125
```

- After running the chain, we can plot the predicted results extracting a random sample on top of the data:



- At least visually, the fit seems to be satisfactory.
- Nevertheles, we can check the posterior PDF of the parameters with a regular “corner plot”:



- Not fully surprinsingly, the most probable values are not fully consistent with the *true* values we used in generating our data.

Modeling the Noise

- Now, instead of assuming that the noise is white, we'll generalize the likelihood function to include covariances between data points. To do this, let's start by re-writing the likelihood function from the previous section as a matrix equation:

$$\ln p(y_n|t_n, \sigma_n^2, \theta) = -\frac{1}{2} r^T K^{-1} r - \frac{1}{2} \ln \det K - \frac{N}{2} \ln 2\pi$$

- where the residual vector is:

$$r = \begin{pmatrix} y_1 - f_\theta(t_1) \\ y_2 - f_\theta(t_2) \\ \vdots \\ y_N - f_\theta(t_N) \end{pmatrix}$$

- And K is the $N \times N$ data covariance matrix (N is the number of data points):

$$K = \begin{pmatrix} \sigma_1^2 & 0 & & 0 \\ 0 & \sigma_2^2 & & 0 \\ & & \ddots & 0 \\ 0 & 0 & & \sigma_N^2 \end{pmatrix}$$

- The fact that K is diagonal is the result of our earlier assumption that the noise was white. If we want to relax this assumption, we just need to start populating the off-diagonal elements of this covariance matrix. If we wanted to make every off-diagonal element of the matrix a free parameter, there would be too many parameters to actually do any inference. Instead, we can simply model the elements of this array as:

$$K_{ij} = \sigma_i^2 \delta_{ij} + k(t_i, t_j)$$

- where δ_{ij} is the Kronecker_delta and $k(\cdot, \cdot)$ is a covariance function that we get to choose. For this exercize, we'll just use the Matérn-3/2 function:

$$k(r) = a^2 (1 + \frac{\sqrt{3r}}{\tau}) \exp(-\frac{\sqrt{3r}}{\tau})$$

- where $r = |t_i - t_j|$, and a^2 and τ are the parameters of the model.

```
begin
    # Now we include a GP in the model
    #
    @model function KernelModel(t,y,ey)
        α ~ Uniform(-5,0)
        l ~ Uniform(-1,1)
        lσ2 ~ Uniform(-10,0)
        lamp ~ Uniform(-10,10)
        llung ~ Uniform(0,10)
        #
        k = exp(lamp) * with_lengthscale(Matern32Kernel(), exp(llung))
        fmod = atomic(GP(t -> fundmod(t,α,l,exp.(lσ2)),k), GPC())
        fx = fmod(t,1e-6 .+ diagm(ey.^2))
        #
        return y ~ fx
    end

    tkmodgp = KernelModel(tdata,ydata,ydataerr);
end;
```

	iteration	chain	α	l	lσ2	lamp	llung	n_steps	altro
1	1001	1	-0.943372	0.0768256	-0.710821	-2.13459	2.17796	1.0	
2	1002	1	-0.934704	0.0686805	-0.859069	-1.15557	2.68882	23.0	
3	1003	1	-0.954116	0.0509867	-0.953501	0.948192	3.43057	7.0	
4	1004	1	-0.941184	0.0503387	-0.795335	0.478023	3.58746	7.0	
5	1005	1	-0.96178	0.0680935	-0.852772	-0.796653	3.06684	7.0	
6	1006	1	-0.967239	0.0553746	-0.86116	-3.31188	1.77754	7.0	
7	1007	1	-0.930161	0.0843447	-0.756586	-1.70355	2.35722	15.0	
8	1008	1	-0.93914	0.090757	-0.882308	-1.61614	2.83327	7.0	
9	1009	1	-0.908486	0.0788334	-0.798897	-1.13614	2.5793	11.0	
10	1010	1	-0.899903	0.0722795	-0.76128	-1.69001	2.65476	7.0	

altro

```
begin
    iterationsgp = 2000
    burninsgp = 1000
    chainsgp = 4

    kchaingp = mapreduce(c -> sample(tkmodgp, NUTS(burninsgp,0.6), iterationsgp),
    chainscat, 1:chainsgp)
end
```

Sampling 100%

Found initial step size
ϵ: 0.2

Sampling 100%

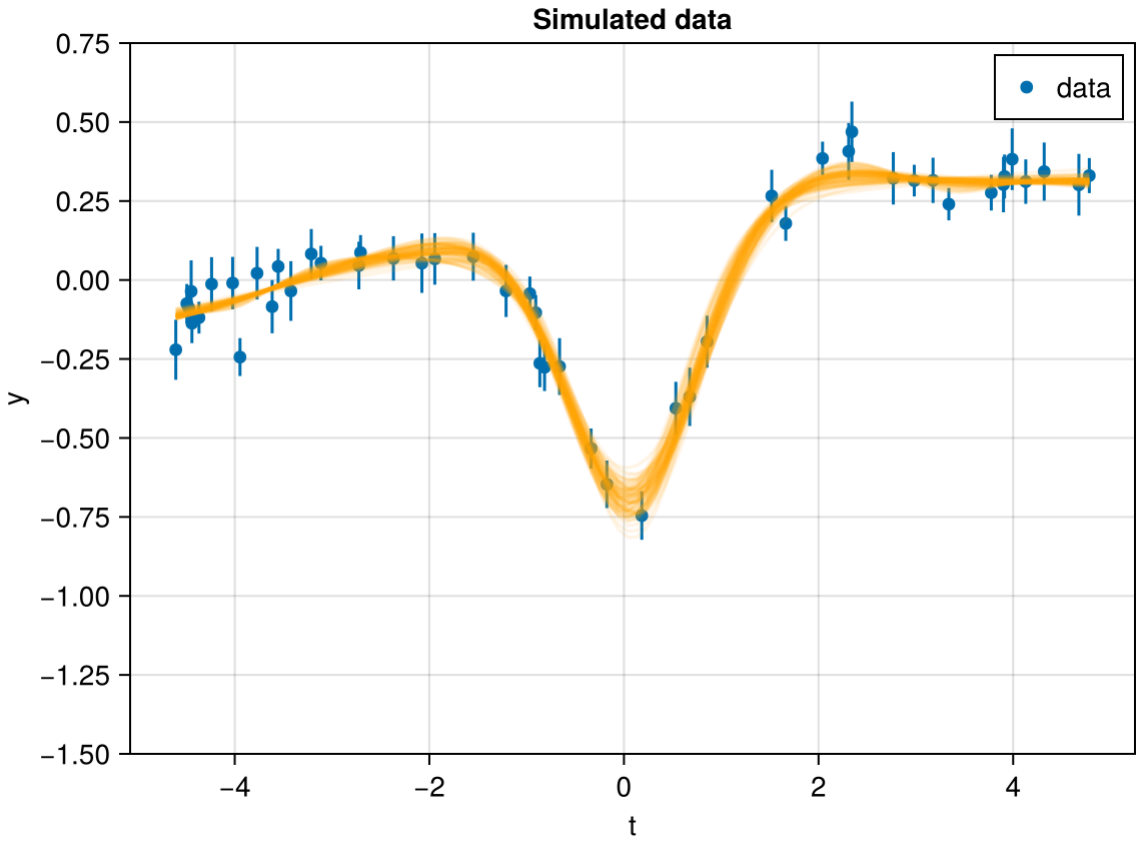
Found initial step size
ϵ: 0.00625

Sampling 100%

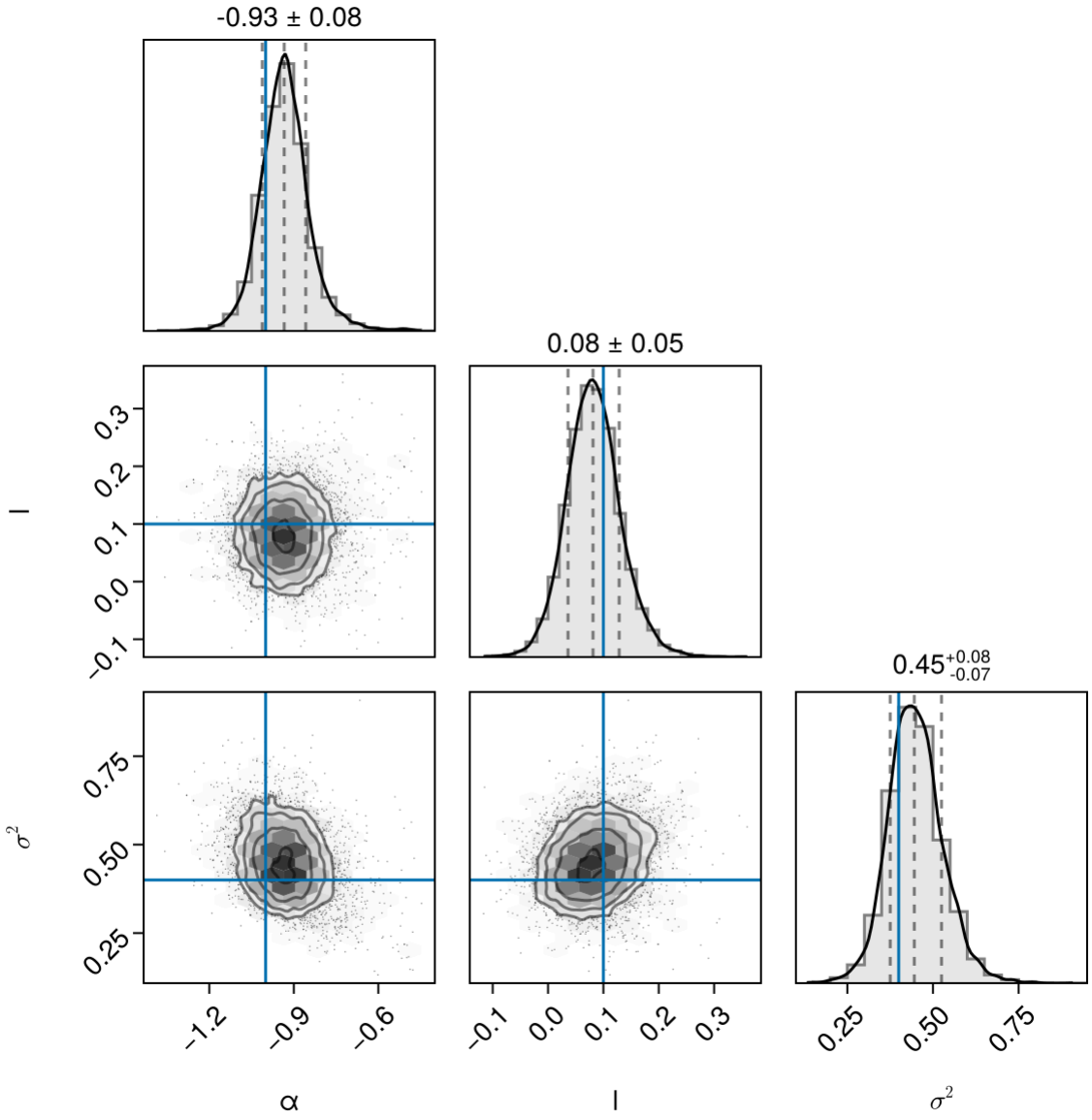
Found initial step size
ϵ: 0.2

Sampling 100%

Found initial step size
ϵ: 0.05



- Again the fit is visually appealing, please note we did not use the same kernel used to generate the data to make the example a bit less contrived.



- And now, finally, the *true* parameters are very close to those obtained by the analysis.

Multivariate GPs

- Since we focus on time domain astronomy, we primarily consider univariate GPs with just time as the input coordinate.
- As a matter of fact, most of our discussion holds even for multivariate datasets.
    - However, when working with univariate GPs, it is relatively straightforward and unambiguous to compute the "distance" between two points $\boldsymbol{t_i}$, and $\boldsymbol{t_j}$: $\boldsymbol{\rho(t_i, t_j) = \tau = |t_i - t_j|}$.
    - For multivariate inputs $\boldsymbol{x}$, more care must be taken to define a sensible distance metric $\boldsymbol{\rho(x_i, x_j)}$.
- A common class of multivariate GP model for time domain astronomy are datasets with multiple parallel covariant time series.
    - For example, multi-band time series produced by surveys like PanSTARRS or LSST, or radial velocity time series with parallel activity indicators.
- One useful way to specify these datasets are to define the inputs as $\boldsymbol{x_i \equiv (t_i, l_i)}$, where $\boldsymbol{t_i}$ is the time of the i-th observation, and $\boldsymbol{l_i}$ is the "label" for which time series the i-th observation is drawn from.

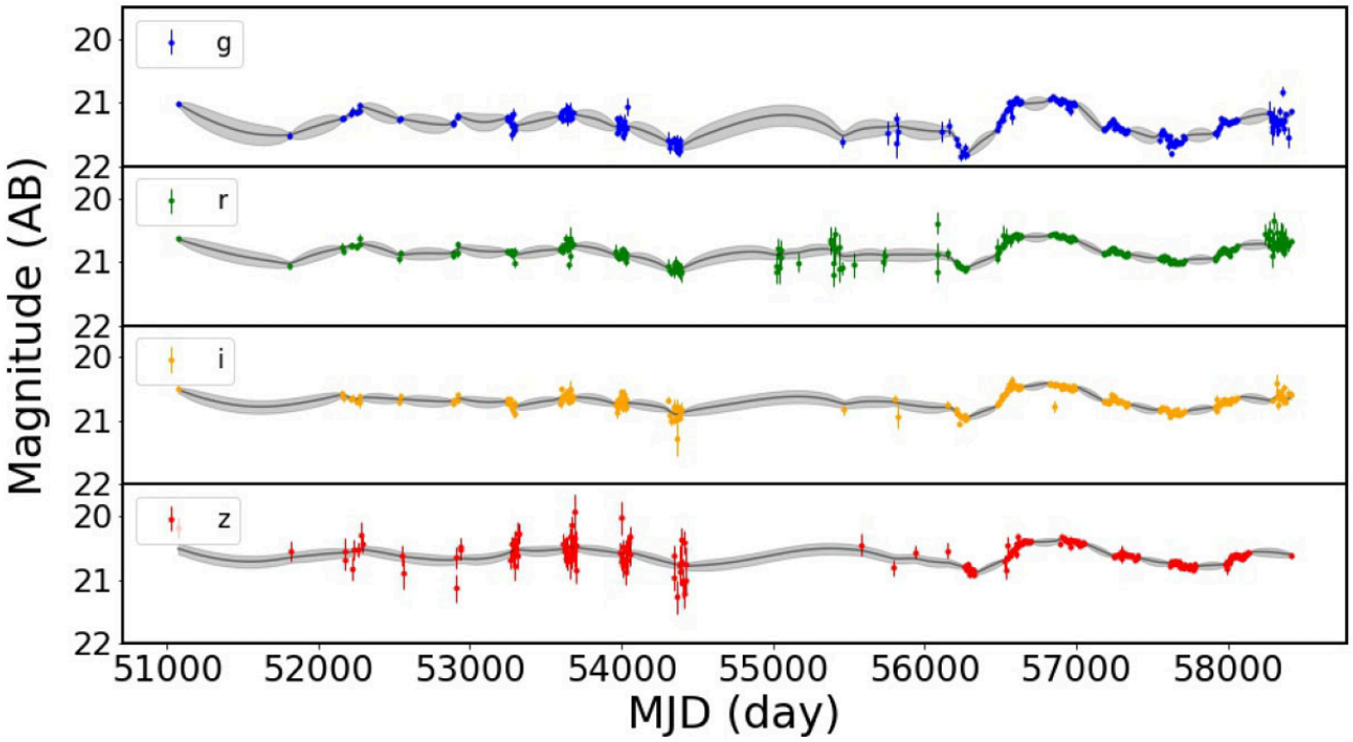
- In this case, one common choice of kernel function is:

$$k(x_i, x_j; \phi) = a_{l_i}^T a_{l_j} k_0(t_i, t_j; \phi)$$

- where $k_0(t_i, t_j; \varphi)$ is a standard one-dimensional kernel, and the set of $\{a_d\}_{d=1}^D$, with D the total number if input time-series, are also hyper-parameters of the model.
- A simple technique implementing this idea is known as *intrinsic model of coregionalization* (Covino et al. 2022), although many other, often more sophisticated, approaches are possible.
- Assuming we have D different input data sets (or bands, as it is often the case in astronomy) the kernel can be defined as: $K_{\text{coreg}} = k(x, x') \cdot B[i, j]$, where x and x' are observations from any input data sets, $k(x, x')$ is a valid covariance function, and B is a $P \times P$ positive-definite matrix:

$$K_{\text{coreg}} = \begin{bmatrix} B_{11}k(X_1, X_1) & \dots & B_{1D}k(X_1, X_D) \\ \vdots & \ddots & \vdots \\ B_{P1}k(X_D, X_1) & \dots & B_{DD}k(X_D, X_D) \end{bmatrix}.$$

- Each output can be seen as linear combination of functions defined by the same covariance $k(x, x')$. The rank of the B matrix defines the number of component in the linear combination. Therefore, an ICM is generated by the products of two covariance functions: *one that models the dependence between the outputs and one that models the input dependence.*
  - If $B_{ij} = 0$ for $i \neq j$ describes the case with independent data for bands i and j.



Reference & Material

Material and papers related to the topics discussed in this lecture.

- Aigrain & Foreman-Mackey (2023) - "Gaussian Process Regression for Astronomical Time Series"
- Covino et al. (2022) - "Detecting the periodicity of highly irregularly sampled light curves with Gaussian processes: the case of SDSS J025214.67-002813.7"

Further Material

Papers for examining more closely some of the discussed topics.

- Covino et al. (2020) - "Looking at Blazar Light-curve Periodicities with Gaussian Processes"
- Deisenroth et al. (2020)- "A Practical Guide to Gaussian Processes"
- O'Sullivan & Aigrain (2024) - "Modelling stochastic and quasi-periodic behaviour in stellar time-series: Gaussian process regression versus power-spectrum fitting"
- Williams & Rasmussen (2005) - "Gaussian Processes for Machine Learning"

Credits

This notebook contains material obtained from https://www.dominodatalab.com/blog/fitting-gaussian-process-models-python and https://george.readthedocs.io/en/latest/tutorials/model/.

Course Flow

	Previous lecture	Next lecture	Course Summary
notebook	Science case about motion sensor data	Science case about CO_2 content in atmosphere	Course Summary
html	Science case about motion sensor data	Science case about CO_2 content in atmosphere	Course Summary

Notebook v1.0.0 - 11 May 2026